ECA5801-IOE AND AI DRIVEN SYSTEM

EXPERIMENT 7:

Optimization of IoE Sensor Data Transmission using Edge-Based Feature Extraction and Bandwidth Reduction

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
# ====================================================
# 1. SIMULATION PARAMETERS
# ====================================================

NUM_SAMPLES = 1000
SAMPLE_INTERVAL = 0.1  # seconds
SIMULATION_TIME = NUM_SAMPLES * SAMPLE_INTERVAL

# Assume each raw sensor sample contains 4 sensor readings
BYTES_PER_READING = 4
NUM_SENSORS = 4

RAW_BYTES_PER_SAMPLE = NUM_SENSORS * BYTES_PER_READING

# ====================================================
# 2. GENERATE IoE SENSOR DATA
# ====================================================
```

```
time = np.arange(NUM_SAMPLES) * SAMPLE_INTERVAL


temperature = 25 + 3 * np.sin(2 * np.pi * 0.1 * time) + np.random.normal(0, 0.3,
NUM_SAMPLES)
humidity = 60 + 5 * np.sin(2 * np.pi * 0.05 * time) + np.random.normal(0, 0.5,
NUM_SAMPLES)
pressure = 1013 + 2 * np.sin(2 * np.pi * 0.02 * time) + np.random.normal(0, 0.2,
NUM_SAMPLES)
light = 500 + 100 * np.sin(2 * np.pi * 0.03 * time) + np.random.normal(0, 5,
NUM_SAMPLES)


raw_data = pd.DataFrame({
    "Time": time,
    "Temperature": temperature,
    "Humidity": humidity,
    "Pressure": pressure,
    "Light": light
})


print("RAW IoE SENSOR DATA")
display(raw_data.head(10))


# ==================================================
# 3. EDGE-BASED FEATURE EXTRACTION
# ==================================================


# Divide sensor data into windows
WINDOW_SIZE = 10
```

```
features = []

for start in range(0, NUM_SAMPLES, WINDOW_SIZE):

    window = raw_data.iloc[start:start + WINDOW_SIZE]

    if len(window) == 0:
        continue

    features.append({
        "Time": window["Time"].mean(),
        "Temperature_Mean": window["Temperature"].mean(),
        "Temperature_Std": window["Temperature"].std(ddof=0),
        "Humidity_Mean": window["Humidity"].mean(),
        "Pressure_Mean": window["Pressure"].mean(),
        "Light_Mean": window["Light"].mean()
    })

processed_data = pd.DataFrame(features)

print("\nEDGE-EXTRACTED FEATURES")
display(processed_data.head(10))

# ==================================================
# 4. RAW VS PROCESSED DATA SIZE
# ==================================================

raw_size_bytes = NUM_SAMPLES * RAW_BYTES_PER_SAMPLE
```

```
# Each window produces 5 numeric features:
# Temperature mean, Temperature standard deviation,
# Humidity mean, Pressure mean, Light mean.
# Assume each numeric feature uses 4 bytes.
NUM_FEATURES = 5
processed_size_bytes = len(processed_data) * NUM_FEATURES * 4

raw_size_kb = raw_size_bytes / 1024
processed_size_kb = processed_size_bytes / 1024

bandwidth_reduction = (
    (raw_size_bytes - processed_size_bytes) / raw_size_bytes
) * 100

print("\nDATA SIZE COMPARISON")
print("--------------------")
print(f"Number of raw samples: {NUM_SAMPLES}")
print(f"Number of processed windows: {len(processed_data)}")
print(f"Raw data size: {raw_size_bytes} bytes ({raw_size_kb:.2f} KB)")
print(f"Processed data size: {processed_size_bytes} bytes ({processed_size_kb:.2f} KB)")
print(f"Bandwidth reduction: {bandwidth_reduction:.2f}%")

# ======================================================
# 5. LATENCY COMPARISON
# ======================================================

# Assumed network transmission rate
BANDWIDTH_BPS = 100000  # 100 kbps
```

```
# Assumed edge feature extraction time per window
EDGE_PROCESSING_MS = 2

# Data transmission time in milliseconds
raw_transmission_ms = (
    raw_size_bytes * 8 / BANDWIDTH_BPS
) * 1000

processed_transmission_ms = (
    processed_size_bytes * 8 / BANDWIDTH_BPS
) * 1000

# Total latency for each approach
raw_total_latency_ms = raw_transmission_ms

processed_total_latency_ms = (
    processed_transmission_ms
    + len(processed_data) * EDGE_PROCESSING_MS
)

print("\nLATENCY COMPARISON")
print("------------------")
print(f"Raw data transmission latency: {raw_total_latency_ms:.2f} ms")
print(f"Processed transmission latency: {processed_transmission_ms:.2f} ms")
print(f"Total edge processing + transmission latency: {processed_total_latency_ms:.2f} ms")

# ================================================
# 6. FEATURE EXTRACTION RESULTS
# ================================================
```

```
print("\nFEATURE EXTRACTION STATISTICS")
print("----------------------------")

print("Temperature mean:", round(raw_data["Temperature"].mean(), 2))
print("Temperature standard deviation:", round(raw_data["Temperature"].std(), 2))
print("Humidity mean:", round(raw_data["Humidity"].mean(), 2))
print("Pressure mean:", round(raw_data["Pressure"].mean(), 2))
print("Light mean:", round(raw_data["Light"].mean(), 2))

# ================================================
# 7. GRAPH 1: RAW VS PROCESSED DATA SIZE
# ================================================

plt.figure(figsize=(8, 5))

plt.bar(
    ["Raw Data", "Processed Data"],
    [raw_size_kb, processed_size_kb]
)

plt.title("Raw vs Edge-Processed Sensor Data Size")
plt.ylabel("Data Size (KB)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# ================================================
# 8. GRAPH 2: BANDWIDTH REDUCTION
# ================================================
```

```
plt.figure(figsize=(7, 5))

plt.bar(
    ["Bandwidth Reduction"],
    [bandwidth_reduction]
)

plt.title("Bandwidth Reduction Using Edge Feature Extraction")
plt.ylabel("Reduction (%)")
plt.ylim(0, 100)
plt.grid(axis="y", alpha=0.3)
plt.show()

# ==================================================
# 9. GRAPH 3: LATENCY COMPARISON
# ==================================================

plt.figure(figsize=(8, 5))

plt.bar(
    ["Raw Transmission", "Edge + Processed Transmission"],
    [raw_total_latency_ms, processed_total_latency_ms]
)

plt.title("Latency Comparison")
plt.ylabel("Total Latency (ms)")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

```
# ==================================================
# 10. GRAPH 4: FEATURE EXTRACTION
# ==================================================

plt.figure(figsize=(10, 5))

plt.plot(
    raw_data["Time"],
    raw_data["Temperature"],
    label="Raw Temperature",
    alpha=0.5
)

plt.plot(
    processed_data["Time"],
    processed_data["Temperature_Mean"],
    label="Extracted Mean Temperature",
    marker="o",
    markersize=3
)

plt.title("Raw Sensor Data vs Extracted Temperature Features")
plt.xlabel("Time (seconds)")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
# ==================================================
# 11. SAVE RESULTS
# ==================================================

raw_data.to_csv("raw_sensor_data.csv", index=False)
processed_data.to_csv("edge_processed_data.csv", index=False)

summary = pd.DataFrame({
    "Metric": [
        "Raw data size (bytes)",
        "Processed data size (bytes)",
        "Bandwidth reduction (%)",
        "Raw transmission latency (ms)",
        "Edge processing + transmission latency (ms)"
    ],
    "Value": [
        raw_size_bytes,
        processed_size_bytes,
        bandwidth_reduction,
        raw_total_latency_ms,
        processed_total_latency_ms
    ]
})

summary.to_csv("experiment_summary.csv", index=False)

print("\nEXPERIMENT COMPLETED SUCCESSFULLY")
print("Files generated:")
print("1. raw_sensor_data.csv")
```

```
print("2. edge_processed_data.csv")
print("3. experiment_summary.csv")

display(summary.round(3))
```

OUTPUT

RAW IoE SENSOR DATA

	Time	Temperature	Humidity	Pressure	Light
0	0.0	25.149014	60.699678	1012.864964	490.460962
1	0.1	25.146892	60.619371	1012.996228	497.582919
2	0.2	25.570306	60.343768	1012.891776	501.700991
3	0.3	26.019053	60.147073	1013.013788	515.090292
4	0.4	25.675824	60.975778	1012.721766	510.315446
5	0.5	25.856810	60.978915	1013.168240	502.733423
6	0.6	26.578138	61.384503	1013.150895	513.715820
7	0.7	26.507568	61.408302	1013.012285	505.419916
8	0.8	26.304419	61.768226	1013.332573	520.436014
9	0.9	26.770248	61.127338	1013.413227	514.527721

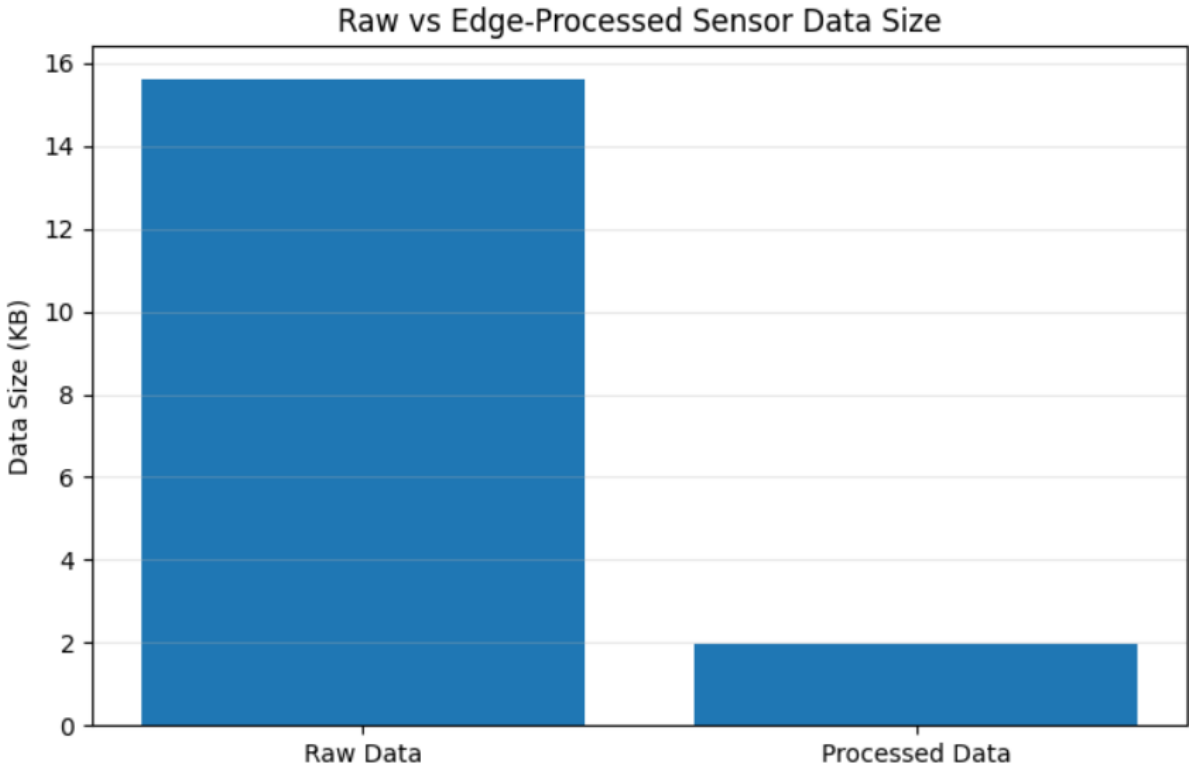
EDGE-EXTRACTED FEATURES

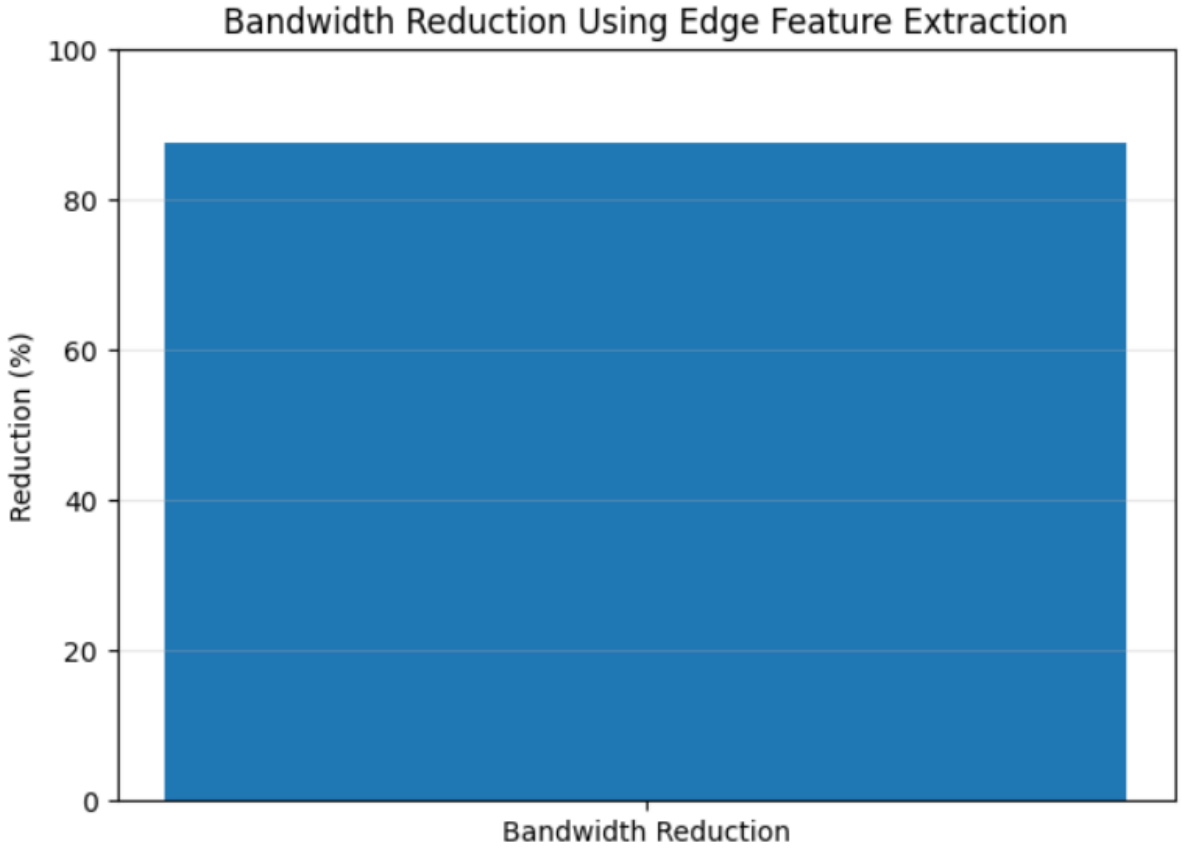
	Time	Temperature_Mean	Temperature_Std	Humidity_Mean	Pressure_Mean	Light_Mean
0	0.45	25.957827	0.549782	60.945295	1013.056574	507.198350
1	1.45	27.094851	0.358113	62.380924	1013.235858	524.329154
2	2.45	27.883371	0.207135	63.664399	1013.611895	544.184810
3	3.45	27.347997	0.518128	64.340493	1013.894106	558.199128
4	4.45	25.923898	0.631270	65.150958	1014.033153	575.033089
5	5.45	24.236424	0.511536	64.961148	1014.218023	583.353116
6	6.45	22.663435	0.328132	64.251632	1014.457547	594.057444
7	7.45	22.035755	0.392319	63.580668	1014.665243	600.805585
8	8.45	22.589405	0.368890	62.464278	1014.654062	598.697184
9	9.45	23.955498	0.519517	60.889446	1014.852416	598.528705

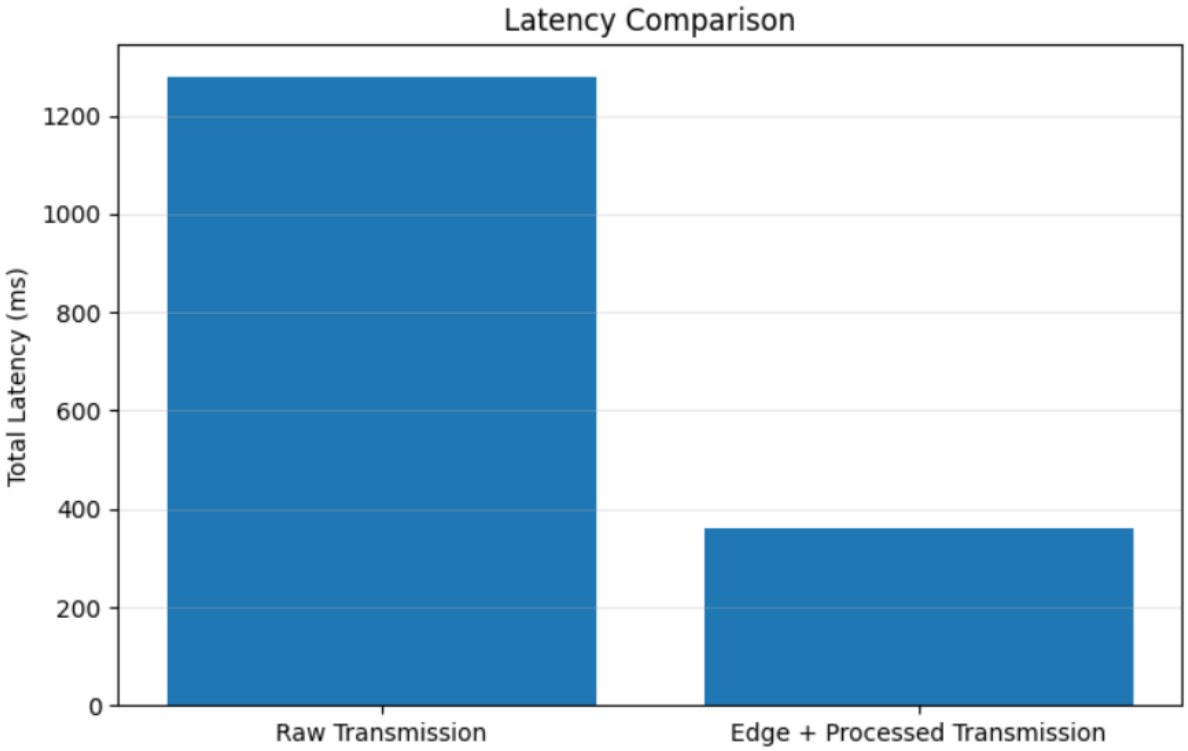
```
DATA SIZE COMPARISON
--------------------
Number of raw samples: 1000
Number of processed windows: 100
Raw data size: 16000 bytes (15.62 KB)
Processed data size: 2000 bytes (1.95 KB)
Bandwidth reduction: 87.50%

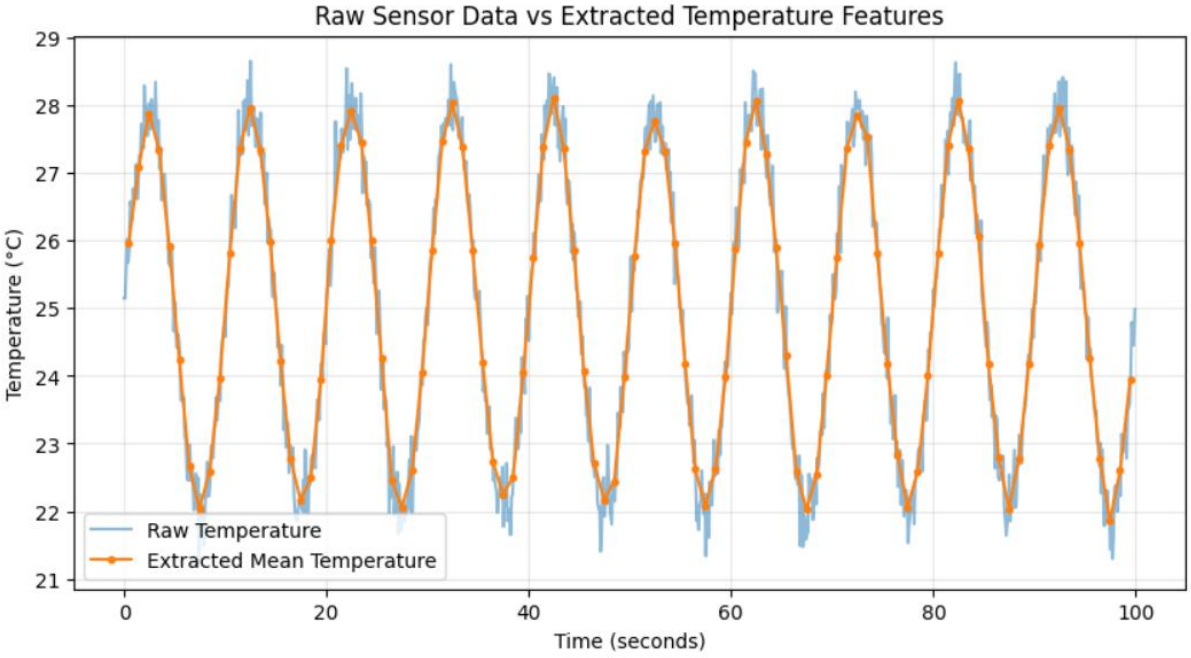
LATENCY COMPARISON
------------------
Raw data transmission latency: 1280.00 ms
Processed transmission latency: 160.00 ms
Total edge processing + transmission latency: 360.00 ms

FEATURE EXTRACTION STATISTICS
-----------------------------
Temperature mean: 25.01
Temperature standard deviation: 2.13
Humidity mean: 60.04
Pressure mean: 1013.0
Light mean: 499.91
```


Raw vs Edge-Processed Sensor Data Size
Data Size (KB)
0
2
4
6
8
10
12
14
16
Raw Data
Processed Data


Bandwidth Reduction Using Edge Feature Extraction
Reduction (%)
0
20
40
60
80
100
Bandwidth Reduction


Latency Comparison
Total Latency (ms)
0
200
400
600
800
1000
1200
Raw Transmission
Edge + Processed Transmission


Raw Sensor Data vs Extracted Temperature Features
Temperature (°C)
21
22
23
24
25
26
27
28
29
0
20
40
60
80
100
Time (seconds)
Raw Temperature
Extracted Mean Temperature